

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования

**«САРАТОВСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ Н. Г. ЧЕРНЫШЕВСКОГО»**

Кафедра математической кибернетики и компьютерных наук

**РЕАЛИЗАЦИЯ АЛГОРИТМА СИММЕТРИЧНОГО ШИФРОВАНИЯ
AES**

КУРСОВАЯ РАБОТА

студента 1 курса 111 группы
направления 02.03.02 — Фундаментальная информатика и информационные
технологии
факультета КНиИТ
Терентьева Даниила Витальевича

Научный руководитель
доцент, к. ф.-м. н.

Н. С. Рыданов

Заведующий кафедрой
доцент, к. ф.-м. н.

С. В. Миронов

Саратов 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Теоретические основы AES	4
2 Математические основы алгоритма AES	6
2.1 Операции в поле $GF(2^8)$	6
2.2 Преобразование SubBytes	6
2.3 Преобразование ShiftRows	6
2.4 Преобразование MixColumns.....	7
3 Реализация AES-128 на языке C++	8
3.1 Функция AddRoundKey	8
3.2 Функция SubBytes.....	8
4 Сравнение режимов шифрования AES	10
ЗАКЛЮЧЕНИЕ	11
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	12
Приложение А Исходный код программы.....	12

ВВЕДЕНИЕ

В современном мире защита информации является одной из ключевых задач информационной безопасности. Для обеспечения конфиденциальности данных широко применяются алгоритмы симметричного шифрования. Одним из наиболее распространённых и надёжных стандартов является алгоритм AES (Advanced Encryption Standard) [1].

Алгоритм AES был принят Национальным институтом стандартов и технологий США (NIST) в 2001 году и заменил устаревший алгоритм DES [1]. AES используется в банковских системах, VPN-сетях, беспроводных протоколах связи, операционных системах и веб-приложениях.

Основным преимуществом AES являются высокая криптостойкость, производительность и возможность эффективной реализации как программными, так и аппаратными средствами [2].

1 Теоретические основы AES

AES представляет собой блочный симметричный алгоритм шифрования [1]. Размер блока данных составляет 128 бит. В зависимости от длины ключа различают следующие версии алгоритма:

- AES-128;
- AES-192;
- AES-256.

В данной работе рассматривается версия AES-128, использующая ключ длиной 128 бит и 10 раундов преобразований.

Каждый раунд AES включает следующие операции:

- SubBytes;
- ShiftRows;
- MixColumns;
- AddRoundKey.

Схема выполнения раундов алгоритма AES представлена на рисунке 1.1.

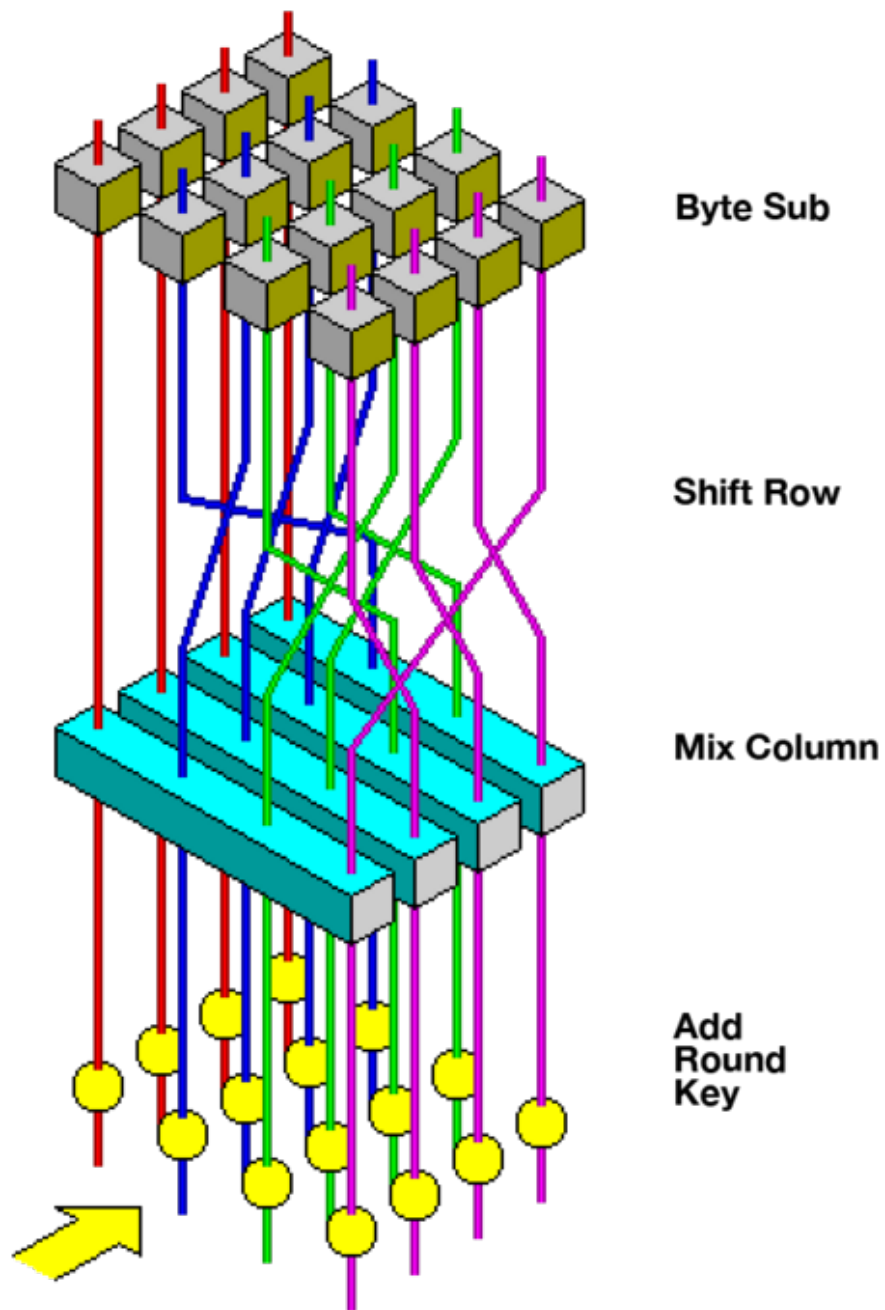


Рисунок 1.1 – Схема раундов алгоритма AES

2 Математические основы алгоритма AES

2.1 Операции в поле $GF(2^8)$

AES выполняет вычисления в конечном поле Галуа $GF(2^8)$ [2]. Каждый байт интерпретируется как полином степени не выше 7.

Сложение элементов поля выполняется по операции XOR:

$$a(x) + b(x) = a(x) \oplus b(x) \quad (2.1)$$

Умножение выполняется по модулю неприводимого полинома:

$$m(x) = x^8 + x^4 + x^3 + x + 1 \quad (2.2)$$

Этому полиному соответствует шестнадцатеричное значение 0x11B:

$$m(x) = 0x11B \quad (2.3)$$

2.2 Преобразование SubBytes

Операция SubBytes выполняет нелинейную замену каждого байта с использованием таблицы замены S-box [1].

Преобразование строится следующим образом:

$$b'_i = A \cdot b_i^{-1} + c \quad (2.4)$$

где:

- b_i^{-1} — мультипликативная обратная величина в поле $GF(2^8)$;
- A — фиксированная матрица аффинного преобразования;
- c — константный вектор.

Преобразование SubBytes повышает криптографическую стойкость алгоритма за счёт нелинейности. Таблица замены S-box представлена на рисунке 2.1.

2.3 Преобразование ShiftRows

Операция ShiftRows выполняет циклический сдвиг строк матрицы состояния влево. Строка с номером r сдвигается на r позиций [1]:

$$s'_{r,c} = s_{r,(c+r) \bmod 4}, \quad r, c \in \{0, 1, 2, 3\} \quad (2.5)$$

	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
00	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
10	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
20	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
30	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
40	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
50	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
60	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
70	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
80	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
90	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A0	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B0	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C0	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D0	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E0	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F0	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

Рисунок 2.1 – Таблица замены S-box алгоритма AES

Таким образом:

- строка 0 не сдвигается;
- строка 1 сдвигается на 1 байт влево;
- строка 2 сдвигается на 2 байта влево;
- строка 3 сдвигается на 3 байта влево.

2.4 Преобразование MixColumns

Операция MixColumns выполняет перемешивание данных внутри каждого столбца матрицы состояния [2].

Преобразование описывается следующим выражением:

$$\begin{bmatrix} b'_0 \\ b'_1 \\ b'_2 \\ b'_3 \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \cdot \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix} \quad (2.6)$$

Все операции выполняются в поле $GF(2^8)$.

3 Реализация AES-128 на языке C++

В данном разделе представлена программная реализация основных операций алгоритма AES-128 на языке C++. Полный исходный код приведён в приложении А.

3.1 Функция AddRoundKey

Функция AddRoundKey выполняет побитовое сложение по модулю два (XOR) между массивом состояния state и раундовым ключом roundKey. Реализация функции представлена в листинге 1.

```
1  #include <stdint>
2
3  void AddRoundKey(uint8_t state[4][4], uint8_t roundKey[4][4])
4  {
5      for (int i = 0; i < 4; i++)
6      {
7          for (int j = 0; j < 4; j++)
8          {
9              state[i][j] ^= roundKey[i][j];
10         }
11     }
12 }
```

Listing 1: Функция AddRoundKey

3.2 Функция SubBytes

Функция SubBytes применяет таблицу нелинейной замены sbox к каждому байту массива состояния, реализуя преобразование из формулы (2.4). Реализация функции представлена в листинге 2.

```

1  #include <stdint>
2
3  const uint8_t sbox[256] = {
4      0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5,
5      0x30, 0x01, 0x67, 0x2b, 0xfe, 0xd7, 0xab, 0x76,
6      0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0,
7      0xad, 0xd4, 0xa2, 0xaf, 0x9c, 0xa4, 0x72, 0xc0,
8      0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc,
9      0x34, 0xa5, 0xe5, 0xf1, 0x71, 0xd8, 0x31, 0x15,
10     0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a,
11     0x07, 0x12, 0x80, 0xe2, 0xeb, 0x27, 0xb2, 0x75,
12     0x09, 0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0,
13     0x52, 0x3b, 0xd6, 0xb3, 0x29, 0xe3, 0x2f, 0x84,
14     0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b,
15     0x6a, 0xcb, 0xbe, 0x39, 0x4a, 0x4c, 0x58, 0xcf,
16     0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85,
17     0x45, 0xf9, 0x02, 0x7f, 0x50, 0x3c, 0x9f, 0xa8,
18     0x51, 0xa3, 0x40, 0x8f, 0x92, 0x9d, 0x38, 0xf5,
19     0xbc, 0xb6, 0xda, 0x21, 0x10, 0xff, 0xf3, 0xd2,
20     0xcd, 0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17,
21     0xc4, 0xa7, 0x7e, 0x3d, 0x64, 0x5d, 0x19, 0x73,
22     0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88,
23     0x46, 0xee, 0xb8, 0x14, 0xde, 0x5e, 0x0b, 0xdb,
24     0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c,
25     0xc2, 0xd3, 0xac, 0x62, 0x91, 0x95, 0xe4, 0x79,
26     0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5, 0x4e, 0xa9,
27     0x6c, 0x56, 0xf4, 0xea, 0x65, 0x7a, 0xae, 0x08,
28     0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6, 0xb4, 0xc6,
29     0xe8, 0xdd, 0x74, 0x1f, 0x4b, 0xbd, 0x8b, 0x8a,
30     0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e,
31     0x61, 0x35, 0x57, 0xb9, 0x86, 0xc1, 0x1d, 0x9e,
32     0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94,
33     0x9b, 0x1e, 0x87, 0xe9, 0xce, 0x55, 0x28, 0xdf,
34     0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42, 0x68,
35     0x41, 0x99, 0x2d, 0x0f, 0xb0, 0x54, 0xbb, 0x16
36 };
37
38 void SubBytes(uint8_t state[4][4])
39 {
40     for (int i = 0; i < 4; ++i)
41     {
42         for (int j = 0; j < 4; ++j)
43         {
44             state[i][j] = sbox[state[i][j]];
45         }
46     }
47 }

```

Listing 2: Функция SubBytes

4 Сравнение режимов шифрования AES

AES может работать в нескольких режимах шифрования [3]. Каждый режим определяет способ применения блочного шифра к данным произвольной длины. Сравнение основных режимов представлено в таблице 4.1.

Таблица 4.1 – Режимы работы AES

Режим	Особенности	Преимущества	Недостатки
ECB	Каждый блок шифруется независимо	Простота реализации	Одинаковые блоки открытого текста дают одинаковый шифротекст
CBC	Каждый блок XOR-ится с предыдущим блоком шифротекста	Высокая криптостойкость	Невозможность параллельного шифрования
CTR	Шифрует последовательный счётчик, результат XOR-ится с текстом	Высокая скорость, поддержка параллелизма	Требует уникального значения поппе для каждого сообщения

Режим ECB не рекомендуется к использованию в криптографических приложениях из-за утечки структуры открытого текста. Режимы CBC и CTR лишены этого недостатка и широко применяются на практике [3].

ЗАКЛЮЧЕНИЕ

В ходе данной работы был рассмотрен алгоритм симметричного блочного шифрования AES-128. Изучены математические основы алгоритма, включая арифметику в поле Галуа $GF(2^8)$ (формула (2.2)), преобразования SubBytes (формула (2.4)), ShiftRows (формула (2.5)) и MixColumns (формула (2.6)).

Реализованы ключевые функции алгоритма на языке C++: AddRoundKey (листинг 1) и SubBytes (листинг 2). Проведено сравнение режимов работы AES (таблица 4.1), выявлены преимущества и недостатки каждого из них.

Алгоритм AES сохраняет актуальность и является одним из наиболее надёжных стандартов шифрования на сегодняшний день [1].

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 FIPS 197: Advanced Encryption Standard (AES) [Электронный ресурс онлайн]. — [Б. м. : б. и.]. — Режим доступа: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf> (дата обращения: 10.05.2026). Загл. с экр. Яз. англ.
- 2 Daemen, Joan. The Design of Rijndael: AES — The Advanced Encryption Standard [Text] / Daemen, Joan and Rijmen, Vincent. — Berlin : Springer, 2002.
- 3 Recommendation for Block Cipher Modes of Operation [Электронный ресурс онлайн]. — [Б. м. : б. и.]. — Режим доступа: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38a.pdf> (дата обращения: 10.05.2026). Загл. с экр. Яз. англ.

ПРИЛОЖЕНИЕ А
Исходный код программы

```
1  #include <cstdint>
2
3  void AddRoundKey(uint8_t state[4][4], uint8_t roundKey[4][4])
4  {
5      for (int i = 0; i < 4; i++)
6      {
7          for (int j = 0; j < 4; j++)
8          {
9              state[i][j] ^= roundKey[i][j];
10         }
11     }
12 }
```

Listing 3: Реализация функции AddRoundKey

```

1  #include <stdint>
2
3  const uint8_t sbox[256] = {
4      0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5,
5      0x30, 0x01, 0x67, 0x2b, 0xfe, 0xd7, 0xab, 0x76,
6      0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0,
7      0xad, 0xd4, 0xa2, 0xaf, 0x9c, 0xa4, 0x72, 0xc0,
8      0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc,
9      0x34, 0xa5, 0xe5, 0xf1, 0x71, 0xd8, 0x31, 0x15,
10     0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a,
11     0x07, 0x12, 0x80, 0xe2, 0xeb, 0x27, 0xb2, 0x75,
12     0x09, 0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0,
13     0x52, 0x3b, 0xd6, 0xb3, 0x29, 0xe3, 0x2f, 0x84,
14     0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b,
15     0x6a, 0xcb, 0xbe, 0x39, 0x4a, 0x4c, 0x58, 0xcf,
16     0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85,
17     0x45, 0xf9, 0x02, 0x7f, 0x50, 0x3c, 0x9f, 0xa8,
18     0x51, 0xa3, 0x40, 0x8f, 0x92, 0x9d, 0x38, 0xf5,
19     0xbc, 0xb6, 0xda, 0x21, 0x10, 0xff, 0xf3, 0xd2,
20     0xcd, 0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17,
21     0xc4, 0xa7, 0x7e, 0x3d, 0x64, 0x5d, 0x19, 0x73,
22     0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88,
23     0x46, 0xee, 0xb8, 0x14, 0xde, 0x5e, 0x0b, 0xdb,
24     0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c,
25     0xc2, 0xd3, 0xac, 0x62, 0x91, 0x95, 0xe4, 0x79,
26     0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5, 0x4e, 0xa9,
27     0x6c, 0x56, 0xf4, 0xea, 0x65, 0x7a, 0xae, 0x08,
28     0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6, 0xb4, 0xc6,
29     0xe8, 0xdd, 0x74, 0x1f, 0x4b, 0xbd, 0x8b, 0x8a,
30     0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e,
31     0x61, 0x35, 0x57, 0xb9, 0x86, 0xc1, 0x1d, 0x9e,
32     0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94,
33     0x9b, 0x1e, 0x87, 0xe9, 0xce, 0x55, 0x28, 0xdf,
34     0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42, 0x68,
35     0x41, 0x99, 0x2d, 0x0f, 0xb0, 0x54, 0xbb, 0x16
36 };
37
38 void SubBytes(uint8_t state[4][4])
39 {
40     for (int i = 0; i < 4; ++i)
41     {
42         for (int j = 0; j < 4; ++j)
43         {
44             state[i][j] = sbox[state[i][j]];
45         }
46     }
47 }

```

Listing 4: Реализация функции SubBytes